Solution Requirements

Date	03 October 2026
Team ID	2
Project Name	FitBuddy – AI Fitness Plan Generator using Gemini Models
Maximum Marks	4 Marks

Define Functional and Non-Functional Requirements:

Step 1: Functional Requirements (FR)

S.No	Requirement Category	Requirement Description	Priority
1	Authentication	End users do not need an account: each plan gets a server-generated user ID (fixes the brief's manually typed ID, which let users overwrite each other). The admin dashboard requires a password (ADMIN_PASSWORD in .env) and uses a signed session cookie.	High
2	Authorization levels	Two roles. User: generate a plan, view own plan and its versions, submit feedback. Admin / coach: view all users with original and latest plans, delete users. Admin routes return 401 / redirect to login without a valid session.	High
3	External interfaces	Google Gemini API via the google-genai SDK: workout model (gemini-3.8-flash) for plans and revisions, light model (gemini-3.5-flash-lite) for tips and as automatic fallback. JSON API documented with Swagger at /docs.	High
4	Transactions processing	Generate plan: validate input → call Gemini with a JSON schema → validate the 7-day plan → save user and plan v1 in one step. Feedback: load latest version → revise → save as a new version (never overwrite). AI errors return HTTP 502 and write nothing.	High
5	Reporting	Result page with day cards, tip and version history; admin dashboard with all users, number of versions and original vs latest plans; JSON endpoint for a user's full plan history; /api/health status report.	Medium
6	Business rules, etc.	Age 13–90, weight 25–300 kg, intensity low/medium/high, experience beginner/intermediate/advanced. Every plan has exactly 7 days and at least one rest/active-recovery day. Extra safety rules for teens (13–17), older adults and heavier users. Feedback ≤ 500 characters.	High

S.No	Requirement Category	Requirement Description	Priority
7	Compliance to laws or regulations	General-fitness information only, with a medical disclaimer on every page. Minimal personal data (name, age, weight); API key stored only in .env, never committed. No calorie-restriction targets or supplement claims in AI output.	Medium
8	Other	Demo mode: with no API key, deterministic built-in plans, tips and revisions keep every feature working for testing and demonstration.	Medium

Step 2: Non-Functional Requirements (NFR)

S.No	NFR Category	Requirement Description	Target Metric / Acceptance Criteria
1	Performance & Speed	Pages and API respond quickly; AI generation shows a loading state.	App stack: avg < 2 s, max < 5 s under 50 users (measured 5.4 ms / 77 ms). Gemini plan ≈ 4–15 s.
2	Scalability	Stateless FastAPI app; database access through SQLAlchemy so SQLite can be swapped for PostgreSQL.	50 concurrent virtual users with 0 failures (load test).
3	Security & Data Privacy	Secrets in .env, admin password + signed cookie, Jinja2 auto-escaping, Pydantic validation, feedback delimited against prompt injection.	API key found in no committed file; invalid input → HTTP 422.
4	Reliability & Availability	One retry on transient errors, automatic fallback model, demo mode, SQLite WAL mode for concurrent writes.	0% error rate in load test; failed AI call never saves data.
5	Usability & Accessibility	Responsive layout (Flexbox, media queries), labelled form fields, HTML5 validation, readable contrast on dark overlay, print stylesheet.	Usable on mobile and desktop; all fields labelled.
6	Other (Maintainability)	Modular files per responsibility, configuration via .env, 245 automated pytest tests (66 test functions).	All 245 tests pass.